Event-Based Retention — Microsoft Purview

Implementation Solution Guide

Version 1.0.0 · 2026-07-07

Contents

Executive Summary 4

1. Solution Overview — Event-Based Retention (Microsoft Purview) 4

What this solution is . . . 5

The event-based mechanic — the core distinction . . . 5

Asset-ID vs keyword triggering — how content is matched to an event . . . 5

What this solution is NOT (the deferred boundary) . . . 6

Why this matters . . . 6

How it works (end to end) . . . 7

The reference design . . . 7

Limitations (be honest with buyers) . . . 8

Naming convention (used in every doc and script) . . . 9

The standard deployment model . . . 9

2. Requirements 10

2.1 Compliance licensing for event-based retention & records . . . 10

2.2 The event-based schedule (input from records & legal) . . . 10

2.3 The system of record and the event-raise process (operational owner) . . . 11

2.4 The asset-ID / keyword matching strategy and its data . . . 11

2.5 Roles for event-based retention & records . . . 11

2.6 In-scope workloads and content state (SPO / OneDrive / Exchange) . . . 12

2.7 Alignment with the broader retention/records program (not duplication) . . . 12

2.8 Automation toolchain for event-based retention . . . 13

2.9 Change control and a documented rollback for event-based retention . . . 13

Global Requirements — Microsoft Purview Solutions 14

1. Microsoft 365 tenant with the required Purview/compliance licensing . . . 14

2. Purview roles (least-privilege) and portal access . . . 14

3. Security & Compliance PowerShell + Graph toolchain . . . 14

4. Data sources & scope in scope . . . 15

5. Compliance, records, privacy & legal stakeholders . . . 15

6. A pilot scope and change control . . . 15

3. Implementation Guide 16

Stage 0 — Connect and confirm prerequisites . . . 16

Stage 1 — Assess (read-only) . . . 16

Stage 2 — Design (map the schedule to event types, labels, and triggers) . . . 17

Stage 3 — Configure (build, staged safe) . . . 17

Stage 4 — Pilot (prove the trigger on a disposable asset) . . . 19

Stage 5 — Validate . . . 19
Stage 6 — Expand (production) . . . 19
Rollback (per stage) . . . 20
5. Testing & Validation 20
6. Troubleshooting 22
7. Best Practices 26
Governance & ownership . . . 26
The raise process is the deliverable . . . 26
Asset-ID / keyword matching . . . 26
Change-safety . . . 27
Operations & evidence . . . 27
Alignment with the broader program . . . 28
8. Deliverables Summary 28
Documentation . . . 28
PowerShell scripts (5) . . . 29
Appendix A — PowerShell Scripts 31

Executive Summary

Most retention is **date-based** — keep a document for a fixed number of years from the day it was created or last modified, a clock the calendar starts on its own. But some of the organization's most sensitive obligations cannot be expressed that way: keep an employee record for *N years after termination*, a contract for *N years after it expires*, a product record for *N years after end-of-life*. On the day the item is created, nobody knows when its clock should start; it starts only when the **event** happens. When that capability is missing, organizations fall back to keeping everything forever — inflating breach and eDiscovery exposure — or deleting on a guessed date, breaking a regulatory or legal obligation. This solution implements **event-based retention in Microsoft Purview** so those obligations are enforced from the event, not a date — configured, piloted, validated, and operable.

Outcome. The three parts of the mechanic, built to work together: **event types** for each real-world event (termination, contract expiry, product end-of-life); **event-based retention (and record) labels** whose clock starts on the event and which sit **pending** until it does; and the **asset-ID / keyword triggering** — the `Compliance Asset Id` on SharePoint/OneDrive documents, keywords for Exchange — that lets a raised retention event start the clock on **exactly** the affected content and nothing else. Crucially, it delivers the piece organizations most often get wrong: a working **raise process** — who raises the event, from which system of record (HR/CLM/PLM), with which asset IDs — so content does not sit pending forever.

Approach. The work is **focused on the event-based mechanic** and **aligned to the organization's broader Microsoft Purview retention/records program** — it reuses that program's event types and labels rather than duplicating them, and it defers records declaration in depth, the file plan, and disposition operations to the org-wide data lifecycle & records product. It implements the **event-based schedule provided by records, compliance, and legal**; it does not author events, retention periods, or legal determinations. Delivery follows a disciplined, pilot-first model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — mapping the schedule to event types, labels, and a matching strategy, publishing labels to a bounded pilot, and **proving the trigger with a test event on a disposable asset** before any real content is affected. Five PowerShell tools (Security & Compliance PowerShell and Graph) handle posture assessment, event-type/ label configuration and event-raising, validation, compliance reporting, and scope/health — read-only or dry-run (`-WhatIf`) by default, with no credentials in code.

Why it is low-risk. Raising a retention event starts real retention clocks, so nothing touches production content until the trigger is proven on a pilot. Assessment, validation, and reporting are read-only; configuration and event-raising are `-WhatIf` by default with an explicit `-Commit`; test events are raised only on disposable assets; every change has a documented rollback, and the **irreversible** actions (an elapsed event-based deletion, a regulatory-record designation) are explicitly flagged as no-rollback and gated on records/legal sign-off. Plan- and cloud-dependent behavior (event-based retention, records, the `Compliance Asset Id` property, Government-cloud portal paths and cmdlet parameters) is flagged for tenant verification rather than assumed. The result: obligations that the calendar cannot express are enforced from the event — nothing kept forever by accident, nothing disposed too early — with the trigger validated and the capability defensible.

1. Solution Overview — Event-Based Retention (Microsoft Purview)

What this solution is

Most retention is **date-based**: keep a document for *N* years from the day it was created, last modified, or labeled — a clock the calendar starts on its own. But some of the most important records must retain from an **event**, not a date: keep an employee record for *N years after termination*, a contract for *N years after it expires*, a product record for *N years after end-of-life*. On the day the item is created nobody knows when its clock should start; it only starts when the event happens. This solution is a repeatable, pilot-first method to implement **event-based retention in Microsoft Purview** — the **event types**, the **event-based retention (and record) labels** that start their clock on an event, and the **asset-ID / keyword triggering** that raises a retention event and starts the clock on exactly the matching content — configured, piloted, validated, and operable, so the capability is in place and defensible.

It is deliberately **focused on the event-based mechanic**. The broader tenant-wide retention program — date-based retention policies, the full records **file plan**, records declaration in depth, and disposition-review operations — is the province of the org-wide **data lifecycle & records management** product; where this solution needs that context it **references** that product rather than rebuilding it. What this solution owns end-to-end is the piece that trips organizations up: getting an event to reliably **start the retention clock on the right content at the right time**.

The event-based mechanic — the core distinction

Event-based retention in Microsoft Purview is built from three parts that must be designed together:

- An **event type** is a named category of event — `Event Type – Employee-Termination`, `Event Type – Contract-Expiration`, `Event Type – Product-EndOfLife`. It is the reusable definition; it does not by itself start any clock.
- An **event-based retention label** is a retention (or record) label whose trigger is set to **"start retention based on an event" **and bound to an event type. Until the event is raised, content with this label sits in a** pending** state — the label is applied, the duration is known, but the clock has not started; the item shows *"retention starts on event."*
- A **retention event** is the act of declaring that the event has happened. It names the event type and identifies the affected content by **asset ID** and/or **keyword**. When Microsoft Purview processes the event, it finds every labeled item whose asset ID / keyword matches and **starts the retention clock** from the event date for exactly those items.

The event type is the vocabulary, the label is the pending instrument, and the retention event is the trigger. All three must line up — same event type on the label and the event, and an asset-ID / keyword strategy that actually matches the content — or the clock never starts.

Asset-ID vs keyword triggering — how content is matched to an event

When a retention event is raised, Microsoft Purview needs to know **which items** the event applies to. There are two matching mechanisms, and choosing correctly is the crux of a working design:

- **Asset ID (SharePoint / OneDrive).** Each document carries a **Compliance Asset Id** property (a

managed-metadata column Purview recognizes). You stamp the relevant asset identifier — an employee ID, a contract number, a product SKU — onto the in-scope documents. When you raise the retention event you supply the same asset IDs, and Purview starts the clock on the documents whose `Compliance Asset Id` matches. This is **precise**: only the tagged items are affected.

- **Keyword (Exchange, and as an alternative).** Instead of (or alongside) asset IDs, a retention event can

match content by **keyword query**. This is how mailbox content is typically matched, and it is broader — anything matching the query is affected — so it must be scoped carefully.

The reference design keys SharePoint/OneDrive event-based retention on **asset IDs** for precision, uses **keywords** where asset-ID stamping is impractical (notably Exchange), and — critically — **defines who stamps the asset ID and when** so the matching data actually exists on the content before an event is ever raised. *(Exact property names, matching behavior, and cmdlet parameters are plan- and version-dependent — verify in your tenant.)*

What this solution is NOT (the deferred boundary)

- **The retention schedule itself is not authored here.** *Which* content is event-based, *which* event

starts its clock, *how long* it then retains, and *whether* it is a record or regulatory record are determined by the organization's **records-management, compliance, privacy, and legal** functions. This solution takes those determinations as input and configures Purview to execute them faithfully; it does not invent event types, retention periods, record status, or legal determinations.

- **Full records management, the file plan in depth, and disposition-review operations** belong to the

org-wide data lifecycle & records product — referenced here, not rebuilt.

- **Legal holds and eDiscovery** (what to preserve for a matter) belong to legal/eDiscovery, not to this

event-based retention configuration.

- **Sensitivity (information-protection) labels, DLP, and the broader Purview data-governance program** are

aligned to, not rebuilt here.

Why this matters

- **The calendar cannot express these obligations.** "Keep 7 years after termination" has no fixed start

date at creation time; only an event-based label plus a raised event can express and enforce it. Without it, organizations fall back to keeping everything forever (over-retention, breach and eDiscovery exposure) or deleting on a guessed date (under-retention, regulatory breach).

- **The event is the whole game — and the usual failure point.** Event types and labels are easy to create;

the hard, high-value work is ensuring the event is actually **raised** with an asset-ID / keyword strategy that matches the right content, wired to the system of record (HR / CLM / PLM) that knows when the event happened. This solution makes that operational, not theoretical.

• **Getting the clock start wrong is expensive both ways.** Start it too early and content is disposed

before the obligation ends; never start it and content is retained forever. Piloting the trigger on disposable test assets, and validating the effective clock start before wiring the real process, is the difference between a defensible control and a latent liability.

How it works (end to end)

1. **Event types** are created in Microsoft Purview → **Records management** → **Events** (or **Data lifecycle

management**) — one per real-world event the schedule names (`New-ComplianceRetentionEventType`).

1. **Event-based labels** are created with a retention trigger of **event** (`New-ComplianceTag

-RetentionType EventAgeInDays -EventType "<event type>"`), carrying the retention action and duration that begin **after** the event, and — where records/legal require — the **record** setting.

1. **Labels are applied** to the in-scope content by **publishing** them for users to apply, **auto-applying**

them by condition, or setting them as a default — scoped to the pilot first.

1. **The asset ID is stamped** on the in-scope SharePoint/OneDrive documents (the **Compliance Asset Id**

property) — the employee ID / contract number / product SKU — so a future event can match them precisely. Where asset-ID stamping is impractical, a **keyword** strategy is defined instead.

1. **Content sits pending.** Labeled items show *"retention starts on event"* — the label and duration are

set, but no clock is running yet.

1. **The event is raised** when it happens (`New-ComplianceRetentionEvent`), naming the event type and

supplying the **asset IDs** and/or **keywords** for the affected content, with the **event date**.

1. **The clock starts.** After service propagation, Purview starts the retention period from the event date

on every labeled item whose asset ID / keyword matched — and only those.

1. **Retention then runs** as configured — retain, then delete or route to disposition review at expiry

(disposition operations deferred to the records product) — with the whole configuration delivered through the house model so it is proven on a bounded pilot before it drives real content.

The reference design

• **Inputs (from records/legal):** the event-based categories — for each, the **event** that starts the

clock, the **retention period after the event**, whether it is a **record / regulatory record**, and the **system of record** that knows when the event occurred (HR / CLM / PLM).

• **Event types:** one named event type per real-world event (`Event Type – Employee-Termination`, `Event

Type – Contract-Expiration`, `Event Type – Product-EndOfLife`).

• **Labels:** one event-based retention/record label per category, bound to its event type, named per the convention, with the post-event action and duration from the schedule.

• **Application:** published and/or auto-apply label policies, scoped to `Scope – EventRetention-Pilot – Static` first.

• **Triggering:** an **asset-ID** strategy for SharePoint/OneDrive (which identifier, who stamps it, when) and a **keyword** strategy where asset IDs are impractical — plus the **raise process** (who raises the event, from which system of record, on what signal).

• **Pilot:** a bounded pilot scope, a disposable test asset, and a **test retention event** to prove the clock starts on exactly the matching content before any real event is raised.

Limitations (be honest with buyers)

• **Event-based retention and records features are plan-dependent.** Event-based retention labels, records declaration, auto-apply, and adaptive scopes generally require an **E5 / E5 Compliance-class** plan or add-on — **verify what your tenant is licensed for.**

• **The clock only starts when an event is raised.** Creating the event type and label does nothing on its own; if no one raises the event (or the asset IDs / keywords don't match), content stays **pending forever** and is never disposed. The **raise process is the deliverable**, not an afterthought.

• **Matching depends on the asset ID actually being on the content.** If the `Compliance Asset Id` was never stamped (or the keyword doesn't match), the event finds nothing. Asset-ID stamping must happen **before** the event, ideally at ingestion.

• **Everything is eventually consistent, not instant.** Label propagation, asset-ID indexing, and event processing run on service timers — expect delays (hours to several days) before a raised event visibly starts clocks. Do not judge a pilot in the first minutes.

• **A raised event that starts a delete clock cannot be un-raised into the past.** Once the clock has started and (for retain-then-delete) elapsed, disposal is real — prove the trigger on disposable test assets only.

• **Records and regulatory records are stronger and (for regulatory) one-way.** Where an event-based label also declares a record, the item is locked; a regulatory record cannot be shortened or removed. Use per records/legal direction only.

• **Commercial vs Government cloud paths differ.** Portal URLs and some feature availability differ in GCC / GCC High / DoD (`*.office365.us`, `Connect-IPPSSession -ConnectionUri`) — **verify the path and feature in your cloud.**

Naming convention (used in every doc and script)

Every artifact this solution creates is named `<Purpose> – <Scope> – <Stage>` and reused verbatim across the docs, scripts, and the Purview admin surfaces:

Artifact	Name
Event type (termination)	`Event Type – Employee-Termination`
Event type (contract)	`Event Type – Contract-Expiration`
Event type (product)	`Event Type – Product-EndOfLife`
Event-based retention label	`Retention Label – Termination-7yr – Pilot`
Event-based record label	`Record Label – Contracts-EventExpiry-7yr – Pilot`
Published label policy	`Label Policy – EventBased-Publish – Pilot`
Auto-apply label policy	`Label Policy – EventBased-AutoApply – Pilot`
Asset-ID property (SPO/OneDrive)	`Compliance Asset Id` (managed-metadata column)
Test retention event	`Retention Event – TEST-Termination – Pilot`
Pilot scope (static)	`Scope – EventRetention-Pilot – Static`
Assessment export	`Event Retention Posture – Tenant – Discovery`
Compliance report	`Event Retention Compliance – Tenant – Report`

When the pilot passes and you expand to production, mint the production equivalents by swapping the `– Pilot` stage for `– Prod` (e.g. `Record Label – Contracts-EventExpiry-7yr – Prod`) — never edit a validated pilot artifact in place. Event types are reusable and typically not stage-suffixed.

The standard deployment model

> ***Assess*** *existing event types, event-based labels, and raised events →* ***Design*** *the event-type / label / asset-ID (or keyword) / raise-process map from the schedule →* ***Configure*** *event types, labels, and pilot-scoped label policies, and stamp asset IDs on pilot content →* ***Pilot*** *by raising a* ***test*** *retention event on a disposable asset on* `Scope – EventRetention-Pilot – Static` *→* ***Validate*** *against explicit pass/fail gates (the clock starts on exactly the matching content) →* ***Expand*** *to production labels, real asset-ID coverage, and the live raise process wired to the system of record.*

Starting (or failing to start) a retention clock on real content is **high-impact**. Every production change goes through change control with a documented rollback, test events are raised only on disposable assets, and nothing drives real content until the gates in Section 5 pass.

2. Requirements

These are the **solution-specific** prerequisites for implementing event-based retention in Microsoft Purview. They build **on** the shared Purview category requirements in `global-requirements-purview.md` (a Microsoft 365 tenant with the required Purview/compliance licensing; least-privilege Purview roles and portal access; the Security & Compliance PowerShell + Graph toolchain; the in-scope data sources/workloads; the compliance/records/privacy/legal stakeholders; and a pilot scope with change control). Do not repeat those — verify them first, then confirm the event-based prerequisites below. Each is stated as **Description / Why / Verify / Implement if missing.**

2.1 Compliance licensing for event-based retention & records

• **Description:** The Microsoft 365 / Microsoft Purview licensing that unlocks **event-based retention labels**, and — where the category is also a record —** records management **(record / regulatory-record labels, the file plan) and** auto-apply **label policies. These advanced retention capabilities are generally in an** E5 / E5 Compliance-class **plan or the equivalent Purview add-on; basic date-based retention is broadly available but is** not** what this solution needs.

• **Why required:** Event-based retention is **plan-gated**. Without the right plan, the "start retention based on an event" trigger, event types, and any record designation in the design cannot be created.

• **Verify:** In the Microsoft 365 admin center → Billing → Licenses, confirm the compliance plan/add-on. In the Microsoft Purview portal, confirm **Data lifecycle management** and (for records categories) **Records management** are visible, and that the **Events** area is present. *Feature-to-plan mapping changes — verify the exact entitlement in your tenant.*

• **Implement if missing:** Have licensing assign/purchase the required compliance plan or add-on to the admin (and any accounts that will manage events and records) before configuration.

2.2 The event-based schedule (input from records & legal)

• **Description:** For each event-based category, the authoritative determinations from records/legal: the **event** that starts the clock (termination, contract expiry, product end-of-life, …), the **retention period after the event**, whether the category is a **record** or **regulatory record**, the applicable **regulatory citation/authority**, and the disposition path at expiry (automatic vs review).

• **Why required:** This solution **implements** these determinations; it does not author them. The event types, labels, and durations are a direct translation of the schedule — without it there is nothing correct to configure, and no event/period may be invented.

• **Verify:** Confirm records/legal has provided (or signed off) the event-based categories naming, for each, the event, the post-event period, record status, citation, and disposition — or at least a pilot subset.

• **Implement if missing:** This is **deferred to the records-management / compliance / legal functions** — request the determinations before configuring. Do not invent events, retention periods, record status, or disposition rules.

2.3 The system of record and the event-raise process (operational owner)

• **Description:** For every event type, the **authoritative system of record** that knows when the event

actually happens (**HR/HRIS** for termination, **contract-lifecycle-management/CLM** for contract expiry, **PLM/ERP** for product end-of-life), and a named **operational owner and process** for **raising the retention event** when the system of record signals the event — including which **asset ID(s) / keyword(s)** identify the affected content.

• **Why required:** An event type and label do nothing until an event is **raised** with matching asset IDs

/ keywords. This is the single most common point of failure: labels sit **pending forever** if no reliable process raises the event. The raise process — manual, scripted, or integrated — is a core deliverable.

• **Verify:** Confirm, per event type, the system of record, the signal that indicates the event, the owner

who raises the event, and the asset-ID / keyword values that will be supplied. Confirm whether raising is manual (portal), scripted (`New-ComplianceRetentionEvent`), or integrated from the system of record.

• **Implement if missing:** Identify the system of record and owner, and define the raise process (Section 3

provides the scripted method) before relying on any event-based label in production.

2.4 The asset-ID / keyword matching strategy and its data

• **Description:** The decision, per event-based category, of **how content is matched** to a retention

event — **asset ID** (the **Compliance Asset Id** property on SharePoint/OneDrive documents) or **keyword** (query, typically for Exchange) — **and** confirmation that the matching data will actually exist on the content: which identifier is used (employee ID / contract number / SKU), **who stamps it, and when** (ideally at ingestion), before any event is raised.

• **Why required:** When an event is raised, Purview matches content by asset ID / keyword; if the

`Compliance Asset Id` was never stamped, or the keyword does not match, the event **finds nothing** and no clock starts. Matching is only as good as the data on the content.

• **Verify:** Confirm the matching mechanism per category; confirm the asset-ID source/format and the

stamping process (or the keyword query and that it reliably matches in-scope content and excludes out-of-scope content); confirm the `Compliance Asset Id` column is available on the in-scope SharePoint/OneDrive locations. *Property behavior and names are version-dependent — verify in your tenant.*

• **Implement if missing:** Define the asset-ID scheme and stamping process (at ingestion where possible),

or define and test the keyword query, so matching data exists on content before events are raised.

2.5 Roles for event-based retention & records

• **Description:** The least-privilege role groups to create and operate event-based retention in Microsoft

Purview: **Records Management** or **Compliance Administrator** / **Compliance Data Administrator** for event types, labels, and label policies; and — because **raising a retention event starts real retention**

clocks — a controlled, named set of accounts authorized to **raise events**. Where categories are records, the corresponding records role and (for disposition) **Disposition Management** apply, deferred to the records product.

- **Why required:** Creating event types, event-based labels, and label policies requires the matching

Purview role; raising events is high-impact and must be restricted to authorized operators to prevent an accidental or premature clock start.

- **Verify:** Microsoft Purview portal → **Roles & scopes** → **Permissions (role groups)** — confirm the

admin's membership; confirm exactly which accounts can raise events.

- **Implement if missing:** Have a Global Admin / Compliance Admin add accounts to the least-privilege role

groups under change control, and formally designate (and limit) the event-raising operators.

2.6 In-scope workloads and content state (SPO / OneDrive / Exchange)

- **Description:** Confirmation of which **workloads and locations** hold the event-based content and that

they support the matching mechanism — **SharePoint/OneDrive** for asset-ID matching (the documents that carry the `Compliance Asset Id`), and **Exchange** where keyword matching is used — plus how labels reach that content (published, auto-applied, or default library label, the last deferred to the SharePoint records product).

- **Why required:** Asset-ID matching is a SharePoint/OneDrive mechanic; keyword matching is typical for

Exchange. Scoping the wrong workloads or locations means labels never reach the content, or events never match it.

- **Verify:** Confirm the in-scope locations per category and that the intended matching mechanism is

supported there; confirm the label-application method per location.

- **Implement if missing:** Define the location scope and application method per category before authoring

labels and policies.

2.7 Alignment with the broader retention/records program (not duplication)

- **Description:** Knowledge of the organization's **existing Microsoft Purview retention/records posture**

— existing retention labels/policies, existing event types, and the org-wide **data lifecycle & records management** program — so this event-based work **aligns** to it (consistent label names, no duplicate event types, understood **retention precedence**) rather than conflicting.

- **Why required:** Event-based labels interact with any other retention touching the same content via

Microsoft's retention-precedence rules; a duplicate event type or an unaware overlapping label produces the wrong effective outcome. Records declaration, the file plan, and disposition operations are owned by the broader program.

• **Verify:** Enumerate existing labels, event types, and policies (`Get-ComplianceTag`, `Get-ComplianceRetentionEventType`, `Get-RetentionCompliancePolicy`) and confirm which are owned by the broader program; note any overlap.

• **Implement if missing:** Engage the retention/records program owners; capture the existing baseline (the assessment script does this) before adding event types or labels.

2.8 Automation toolchain for event-based retention

• **Description:** The **Security & Compliance PowerShell** endpoint (`Connect-IPPSSession` from the **ExchangeOnlineManagement** module) for event types, labels, label policies, and raising events (`Get-/New-/Set-ComplianceRetentionEventType`, `New-ComplianceTag`, `New-RetentionCompliancePolicy`/`Rule`, `Get-/New-ComplianceRetentionEvent`), and **Microsoft Graph PowerShell** for supporting reads — a modern-auth / MFA-capable admin.

• **Why required:** The provided scripts and the configuration/raise steps run through Security & Compliance PowerShell (and Graph for supporting reads); both must connect.

• **Verify:** `Connect-IPPSSession` succeeds and `Get-ComplianceTag` / `Get-ComplianceRetentionEventType` return without error; `Connect-MgGraph` succeeds where used. In Government clouds, confirm the correct `-ConnectionUri` / endpoint.

• **Implement if missing:** Install the **ExchangeOnlineManagement** and **Microsoft.Graph** modules and confirm modern-auth sign-in before running steps. Never store credentials in scripts.

2.9 Change control and a documented rollback for event-based retention

• **Description:** An approved change window and a **rollback plan** specific to event-based retention: how to unpublish/disable a label policy, how to remove a non-record event-based label, and the explicit note that **a raised event that has already started (and, for retain-then-delete, elapsed) a delete clock, and any regulatory-record designation, cannot be rolled back**.

• **Why required:** Raising an event and starting a retention clock on real content is high-impact; a documented, rehearsed rollback is the safety net for the reversible parts (labels, policies) and forces a conscious decision on the irreversible parts (an elapsed event-based deletion, a regulatory record).

• **Verify:** Confirm a change ticket, an approver, and a written rollback covering label-policy disable, label removal, and the explicit no-rollback flag on any regulatory-record category and on any real (non-test) event that starts a delete clock.

• **Implement if missing:** Author the rollback (Section 3 provides the steps), get change-control sign-off, and explicitly flag irreversible categories/actions as **no-rollback** before applying them. Raise test events only on disposable assets.

Global Requirements — Microsoft Purview Solutions

These requirements apply to **every** solution in the **Purview** category (implementing and managing Microsoft Purview, sensitivity & retention labels, DLP, data lifecycle & records, insider risk, communication compliance, audit, data security posture management, and data governance). Verify each before starting a solution's implementation steps.

> ***Vendor boundary (applies to every Purview solution).*** *The **Microsoft Purview portal** (purview.microsoft.com), the **Microsoft 365 admin center**, **Security & Compliance PowerShell** (`Connect-IPPSSession`), and **Microsoft Graph** are authored at exact, current click-by-click / cmdlet detail from Microsoft's public documentation. Microsoft Purview features are heavily **licensing-/plan-dependent** (many require **Microsoft 365 E5** or the **E5 Compliance** add-on; some are in preview) and the portal is **actively evolving** — capability availability and portal paths are **cited or flagged "verify your licensing/tenant"**, never asserted or invented. **Regulatory, records-retention, and legal determinations** (retention schedules, records file plans, legal holds, insider-risk case actions, regulatory applicability) are **deferred to the organization's compliance, records, privacy, and legal functions** — the solutions provide the technical configuration method and evidence, not the determination. Automation is **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets.*

1. Microsoft 365 tenant with the required Purview/compliance licensing

• **Description:** A Microsoft 365 tenant with the **compliance licensing** the capability requires

(E3 for baseline; **E5 / E5 Compliance** for advanced DLP, auto-labeling, insider risk, communication compliance, Audit Premium, DSPM, etc.).

• **Why required:** Most Purview capabilities are plan-gated; the feature won't be available without the

right plan.

• **How to verify:** In the M365 admin center confirm the subscription/add-ons; check the feature is

visible in the Purview portal.

• **How to implement if missing:** Assign/purchase the required licensing before starting.

2. Purview roles (least-privilege) and portal access

• **Description:** Access with the least-privilege **Purview / compliance role** the solution names

(e.g. Compliance Administrator, Compliance Data Administrator, Information Protection, Records Management, Insider Risk, eDiscovery, Audit roles), with Global Administrator only where required.

• **Why required:** Purview uses fine-grained role groups; configuration requires the matching role.

• **How to verify:** Purview portal → **Roles & scopes / Permissions** — confirm the role membership.

• **How to implement if missing:** Assign the least-privilege role group under change control.

3. Security & Compliance PowerShell + Graph toolchain

- **Description:** A workstation with **Security & Compliance PowerShell** (`Connect-IPPSSession` via the

ExchangeOnlineManagement module) and **Microsoft Graph PowerShell**, modern-auth / MFA-capable admin.

- **Why required:** Configuration, automation, and the provided scripts run through these.
- **How to verify:** `Connect-IPPSSession` and `Connect-MgGraph` succeed with the admin account.
- **How to implement if missing:** Install the modules and confirm modern-auth sign-in before running steps.

4. Data sources & scope in scope

- **Description:** The **workloads and data sources** in scope (Exchange, SharePoint, OneDrive, Teams,

endpoints/devices, and — for data governance — the broader data estate) and a **data-classification / sensitivity** intent to drive labels, DLP, and retention.

- **Why required:** Purview policies key off the in-scope workloads and the classification scheme.
- **How to verify:** Confirm the in-scope workloads and the classification/sensitivity approach with

stakeholders.

- **How to implement if missing:** Define the scope and classification intent before authoring policies.

5. Compliance, records, privacy & legal stakeholders

- **Description:** Engagement with the **compliance, records-management, privacy, security, and legal**

stakeholders whose policies the configuration must satisfy (retention schedules, records file plans, regulatory obligations, insider-risk and communication-compliance policy, legal hold).

- **Why required:** Purview configuration must align to enterprise policy and legal/records requirements,

not technical defaults; several capabilities have HR/legal sensitivity.

- **How to verify:** Confirm the applicable policies, the records schedule, and named approvers.
- **How to implement if missing:** Identify the owners and capture the requirements before configuring.

6. A pilot scope and change control

- **Description:** A **pilot scope** (users/sites/policies in simulation or a limited ring) to validate

on before tenant-wide enforcement, and a **change-control** process — most Purview policies support a **simulation / test / disabled** mode that MUST be used before enforcement.

- **Why required:** Every solution follows an **Assess → Design → Configure → Pilot (simulate) → Validate

→ Expand (enforce)** model; DLP, labels, retention, and insider-risk changes are high-impact.

- **How to verify:** Confirm a named pilot scope, that simulation/test mode is available, and a change

window/approval path.

- **How to implement if missing:** Define the pilot and agree change control before enforcing anything.

*The deployment model across every Purview solution is the same: **Assess** → **Design** → **Configure** → **Pilot (simulate/test)** → **Validate** → **Expand (enforce)**, with explicit pass/fail gates and a documented rollback. Automation is delivered as **PowerShell** (Security & Compliance PowerShell / Graph) — never with hardcoded credentials or secrets. Regulatory/records/legal determinations are deferred to the relevant governance functions.*

3. Implementation Guide

This is the ordered, pilot-first method to implement event-based retention in Microsoft Purview. It follows the house model — **Assess** → **Design** → **Configure** → **Pilot** → **Validate** → **Expand** — and every artifact is named per the convention in Section 1 and reused verbatim in the scripts. **Raising a retention event starts real retention clocks** and is high-impact: labels are published to the **pilot scope only**, test events are raised on **disposable assets only**, and nothing drives real content until the Section 5 gates pass. The **event-based schedule and all legal/records determinations are inputs from records/legal** — this guide implements them, it does not author them.

> ***Portal paths, property names, and cmdlet parameters evolve and are cloud-dependent.*** *Paths and parameters below are Microsoft Purview portal (`purview.microsoft.com`) / Security & Compliance PowerShell as of authoring; in GCC/GCC High/DoD the URL and some features differ (`*.office365.us`, `Connect-IPPSSession -ConnectionUri`). **Verify paths, the** `Compliance Asset Id` **property, and cmdlet parameters in your tenant/cloud** before relying on them. Where a step is plan-dependent, it is flagged.*

Stage 0 — Connect and confirm prerequisites

1. Confirm Section 2 prerequisites: licensing (2.1), the event-based schedule (2.2), the system of record and

raise process (2.3), the asset-ID/keyword strategy (2.4), roles incl. event-raise authorization (2.5), in-scope workloads (2.6), existing posture (2.7), and change control (2.9).

1. Connect the toolchain from an MFA-capable admin workstation:

```
Connect-IPPSSession                 # Security & Compliance PowerShell (add -ConnectionUri for Gov clou
Connect-MgGraph -Scopes "Directory.Read.All"   # supporting reads only
```

1. Run `Resolve-EventRetentionScope.ps1` to confirm module/connection health, report the detected cloud,

confirm the event/label cmdlets are reachable, and resolve the pilot scope before touching anything. **Pass/fail:** connections succeed, the expected cloud is reported, event-based retention cmdlets are reachable, and `Scope – EventRetention-Pilot – Static` resolves.

Stage 1 — Assess (read-only)

1. Run `Get-EventRetentionPosture.ps1` to produce `Event Retention Posture – Tenant – Discovery`: existing

event types (`Get-ComplianceRetentionEventType`), existing **event-based labels** (`Get-ComplianceTag` filtered to an `Event*` retention type, with record/regulatory flags and their `EventType`), the **label policies** that publish/auto-apply them (`Get-RetentionCompliancePolicy`/`-Rule`), and any **retention events** already raised (`Get-ComplianceRetentionEvent`).

1. Review the baseline with the retention/records program owners (2.7): which event types already exist (do

not duplicate), which labels are owned by the broader program, and any overlap to reason through against retention precedence.

1. **Pass/fail (Assess gate):** a complete inventory export exists; existing event types/labels are

identified so none are duplicated; no destructive action was taken (read-only).

Stage 2 — Design (map the schedule to event types, labels, and triggers)

Translate the records/legal event-based schedule into a concrete artifact map. Produce a design table with, for each event-based category:

- **Event → event type:** the real-world event and its named `Event Type – …`.
- **Post-event action + duration:** retain / retain-then-delete, and the period that begins **after** the

event.

- **Record status:** none / record / **regulatory record** (regulatory only where records/legal require;

records declaration mechanics deferred to the records product).

- **Matching mechanism + data: asset ID** (which identifier — employee ID / contract number / SKU — and

who stamps the `Compliance Asset Id`, when) or **keyword** (the query, and where it applies).

- **Locations + application:** which workloads (SPO / OneDrive / Exchange) and how the label is applied

(published / auto-apply / default).

- **Raise process:** the system of record, the signal, the owner who raises the event, and the asset-ID /

keyword values that will be supplied.

Name every artifact now (Section 1 table) and record the **pilot scope** `Scope – EventRetention-Pilot – Static`. **Pass/fail (Design gate):** every event-based category maps to a named event type, a named label (with post-event action, duration, record status), a defined matching mechanism **with a stamping/keyword plan**, locations/application, and a raise process; records/legal has signed off the mapping (especially any regulatory-record categories).

Stage 3 — Configure (build, staged safe)

Build in this order. **Configuration scripts default to `-WhatIf`/dry-run and only write with `-Commit`.**

3a. Event types

Create the event type per category (S&C PowerShell). `Set-EventTypesAndLabels.ps1` stages these with `-WhatIf`; with `-Commit` it runs:

```
New-ComplianceRetentionEventType -Name "Event Type – Employee-Termination"
New-ComplianceRetentionEventType -Name "Event Type – Contract-Expiration"
New-ComplianceRetentionEventType -Name "Event Type – Product-EndOfLife"
```

Reuse an existing event type from the assessment rather than creating a duplicate.

3b. Event-based labels

Create each event-based retention/record label, bound to its event type, via `Set-EventTypesAndLabels.ps1` (which wraps `New-ComplianceTag` with a `-WhatIf` default). The retention trigger is an **event** type (`-RetentionType EventAgeInDays`) and the duration is the period that begins **after** the event. Examples of the intent the script commits with `-Commit`:

```
# Event-based retention label (retain N years after the event, then delete)
New-ComplianceTag -Name "Retention Label – Termination-7yr – Pilot" `
  -RetentionAction KeepAndDelete -RetentionDuration 2555 `
  -RetentionType EventAgeInDays -EventType "Event Type – Employee-Termination"

# Event-based RECORD label (records-grade; records mechanics deferred to the records product)
New-ComplianceTag -Name "Record Label – Contracts-EventExpiry-7yr – Pilot" -IsRecordLabel $true `
  -RetentionAction KeepAndDelete -RetentionDuration 2555 `
  -RetentionType EventAgeInDays -EventType "Event Type – Contract-Expiration"
```

> ***Regulatory records are irreversible.*** *Where a category is a regulatory record, the config script refuses to set the regulatory flag without an explicit confirmation switch and a records/legal sign-off reference. Do not apply a regulatory event-based label beyond a disposable test item in pilot. *(Verify the exact `-RetentionType` value and record parameters for event-based labels in your tenant/module version.)**

3c. Label policies (publish / auto-apply to the pilot)

Publish the labels to pilot users, or auto-apply by condition, scoped to the **pilot only**:

```
# Publish event-based labels for users to apply (pilot scope)
New-RetentionCompliancePolicy -Name "Label Policy – EventBased-Publish – Pilot" `
  -SharePointLocation "https://contoso.sharepoint.com/sites/EventRetentionPilot" `
  -OneDriveLocation "https://contoso-my.sharepoint.com/personal/pilotuser_contoso_com"
New-RetentionComplianceRule -Policy "Label Policy – EventBased-Publish – Pilot" `
  -PublishComplianceTag "Retention Label – Termination-7yr – Pilot"

# Auto-apply an event-based label by condition (pilot scope) — condition from the schedule, not invented
New-RetentionCompliancePolicy -Name "Label Policy – EventBased-AutoApply – Pilot" `
  -SharePointLocation "https://contoso.sharepoint.com/sites/EventRetentionPilot"
New-RetentionComplianceRule -Policy "Label Policy – EventBased-AutoApply – Pilot" `
  -ApplyComplianceTag "Record Label – Contracts-EventExpiry-7yr – Pilot" `
  -ContentMatchQuery '"master services agreement"'
```

(SharePoint default-library-label application is deferred to the SharePoint records/retention product.)

3d. Stamp the asset ID on pilot content (the matching data)

For asset-ID categories, ensure the `Compliance Asset Id` managed-metadata property is present on the in-scope SharePoint/OneDrive documents and **stamp the identifier** (employee ID / contract number / SKU) onto the pilot documents — via the library UI, a content-type default, or an automated stamping process at ingestion. This is the data a future event matches on; without it the event finds nothing. For **keyword** categories, confirm the query matches the pilot content (and excludes out-of-scope content). **Pass/fail (Configure gate):** every designed event type and label exists, label policies are pilot-scoped, and the matching data (asset ID stamped, or keyword validated) is present on the pilot content; no label

reaches content outside the pilot.

Stage 4 — Pilot (prove the trigger on a disposable asset)

1. Confirm labeled pilot content is in the **pending** state: it shows *"retention starts on event"* — the label and duration are set, but no clock is running yet.

1. Stamp a **disposable test document** with a test asset ID (e.g. `TEST-ASSET-001`) and apply the event-based label to it.

1. Raise a **test retention event** for the category, on the disposable asset only, via `Set-EventTypesAndLabels.ps1 -RaiseEvent` (dry-run by default; `-Commit` calls `New-ComplianceRetentionEvent`):

```
New-ComplianceRetentionEvent -Name "Retention Event – TEST-Termination – Pilot" `
  -EventType "Event Type – Employee-Termination" `
  -EventTags "Retention Label – Termination-7yr – Pilot" `
  -AssetId "TEST-ASSET-001" -EventDateTime (Get-Date)
```

(Use the asset ID for asset-ID categories, or the keyword parameter for keyword categories — verify the exact parameter names in your tenant/module version.)

1. Allow for **propagation** (hours to days — do not judge in minutes). Then run the Stage 5 validation to confirm the clock started on **exactly** the matching asset and on nothing else.

1. **Pass/fail (Pilot gate):** the test event starts the retention clock on the matching disposable asset from the event date, un-matched content stays pending, and no content outside the pilot scope is affected.

Stage 5 — Validate

Run `Test-EventRetentionConfig.ps1` (read-only) to confirm every designed event type exists, each label uses an event trigger bound to the correct event type, label policies target the pilot scope, and asset-ID / keyword readiness is in place; then run `Get-EventRetentionReport.ps1` to produce `Event Retention Compliance – Tenant – Report` and work the Section 5 test procedures. Every gate has an explicit pass/fail. Do not expand until all pass.

Stage 6 — Expand (production)

1. Mint the **production** labels by swapping `– Pilot` for `– Prod` (never edit a validated pilot artifact in place): e.g. `Retention Label – Termination-7yr – Prod`, `Label Policy – EventBased-Publish – Prod`. Event types are reusable and carried forward.

1. Broaden the label policies to the full in-scope locations, and roll out **asset-ID stamping at ingestion** (or the validated keyword strategy) across production content so future events match.

1. **Wire the real raise process** to the system of record (2.3): the owner raises the production event (`New-ComplianceRetentionEvent`) when HR/CLM/PLM signals it, supplying the real asset IDs / keywords and the real event date. Stage the rollout in **rings** (by department/site/category) where possible.

1. **Pass/fail (Expand gate):** production labels apply across the in-scope workloads, asset-ID / keyword

matching data is present on production content, the raise process is operable and tested against the system of record, the compliance report shows expected coverage, and no unexpected retention conflicts appear.

Rollback (per stage)

- **Label policies (reversible):** unpublish/disable a label policy by disabling or removing its rule (`Set-RetentionComplianceRule`), or reduce its locations. This stops the label reaching new content; it does not un-apply labels already on items.
- **Event-based labels (reversible if not a record):** remove a non-record event-based label from items and delete the unused label; a published label can be unpublished; auto-apply can be disabled.
- **Records (partly reversible):** a **non-regulatory** record can be **un-declared** by a records manager (auditable) — do this only per records/legal direction; the mechanic is the records product's.
- **A raised event before its clock elapses (mitigable):** while a `KeepAndDelete` clock is still running you can extend, relabel, or remove the label to change the outcome — treat this as mitigation, not clean rollback, and act before the delete date.
- **IRREVERSIBLE — no rollback:** an **elapsed event-based deletion** (content already disposed) and a **regulatory-record** designation (cannot be removed/shortened). These are gated on records/legal sign-off and must never be exercised beyond a disposable test item in pilot.

Every production change carries a change ticket, an approver, and this rollback written out for the specific categories in scope, with irreversible categories/actions explicitly flagged **no-rollback**. Raise real (non-test) events only through the authorized, change-controlled process.

5. Testing & Validation

Nine procedures, each with an explicit **pass/fail**. Run them at the **Pilot** and **Expand** gates. Event-based retention is **eventually consistent** — allow for service propagation (hours to a few days) after applying a label, stamping an asset ID, or raising an event before declaring a fail. Raise events only on **disposable test assets**; never raise a real event that starts a delete clock as a test, and never apply a regulatory-record label to content you are not prepared to keep permanently.

T1 — Toolchain & scope health

- **Do:** Run `Resolve-EventRetentionScope.ps1`.
- **Pass:** `Connect-IPPSSession` (and Graph where used) succeed; the correct cloud is reported; the event-based retention cmdlets are reachable; `Scope – EventRetention-Pilot – Static` resolves.
- **Fail:** any connection fails, the wrong cloud/endpoint is reported, the event cmdlets are absent (plan/role), or the scope does not resolve.

T2 — Baseline inventory captured (read-only)

- **Do:** Run `Get-EventRetentionPosture.ps1`.
- **Pass:** `Event Retention Posture – Tenant – Discovery` lists existing event types, existing event-based

labels (with their `EventType` and record/regulatory flags), their label policies, and any raised events; existing artifacts are identified so none are duplicated.

- **Fail:** the inventory is incomplete, or the script wrote any change (it must be read-only).

T3 — Event types created as designed

- **Do:** `Get-ComplianceRetentionEventType` for each designed event type.
- **Pass:** each event type exists with the designed name (`Event Type – Employee-Termination`, etc.), and no

duplicate of an existing event type was created.

- **Fail:** an event type is missing, misnamed, or duplicates one already present.

T4 — Event-based labels bound to the correct event type

- **Do:** `Get-ComplianceTag` for each designed label; inspect its retention type and event type.
- **Pass:** each label exists with an **event** retention trigger (`RetentionType` is an `Event*` type),

bound to the **correct** `EventType`, with the designed post-event action and duration, and the correct record/regulatory flag — matching the Stage 2 design.

- **Fail:** any label is missing, uses a date-based trigger instead of an event, is bound to the wrong event

type, or its action/duration/record status differs from design.

T5 — Label publish / auto-apply reaches the pilot only

- **Do:** As a pilot user, confirm the published event-based labels are selectable (SharePoint/OneDrive);

create/identify content matching an auto-apply condition and wait for propagation.

- **Pass:** published labels appear for pilot users only; auto-apply applies the designed label to matching

pilot content; no content outside the pilot scope is affected.

- **Fail:** labels do not appear, apply to the wrong scope, or the wrong label is auto-applied.

T6 — Labeled content is in the pending ("starts on event") state

- **Do:** On a labeled pilot item, inspect its retention state before any event is raised.
- **Pass:** the item shows retention **pending** — *"retention starts on event"* — with the label and

duration set but no clock running.

- **Fail:** the clock has already started (mis-configured as date-based), or the item shows no event-based

retention at all.

T7 — Asset-ID / keyword matching data is present on content

- **Do:** For asset-ID categories, confirm the `Compliance Asset Id` property is stamped on the in-scope

pilot documents with the correct identifier; for keyword categories, confirm the query matches the in-scope content and excludes out-of-scope content.

- **Pass:** the matching data exists on exactly the in-scope content — asset IDs stamped as designed, or the

keyword query verified to match in-scope and not out-of-scope.

- **Fail:** the `Compliance Asset Id` is missing/blank on in-scope content, or the keyword query matches the

wrong (or no) content.

T8 — Raising a test event starts the clock on exactly the matching content

- **Do:** Stamp a disposable test asset (`TEST-ASSET-001`), apply the event-based label, and raise a **test**

retention event on that asset only (`Set-EventTypesAndLabels.ps1 -RaiseEvent -Commit`, or portal). Wait for propagation.

- **Pass:** after propagation, the retention clock starts from the event date on the matching disposable

asset **and only that asset**; un-matched labeled content stays pending; the event is visible via `Get-ComplianceRetentionEvent`.

- **Fail:** the clock never starts, starts on the wrong/too-broad content, or starts at the wrong date.

T9 — Reporting reflects reality

- **Do:** Run `Test-EventRetentionConfig.ps1` (read-only validation) and `Get-EventRetentionReport.ps1`.

- **Pass:** validation confirms event types exist, labels use the correct event triggers/linkage, and label

policies are pilot-scoped; the report's counts of event types, event-based labels, and raised events, and its gap flags (labels without an event type, event types with no label, raised-but-unmatched events) are consistent with T3–T8.

- **Fail:** validation or the report disagrees with the observed state, or omits configured artifacts.

Overall gate. The pilot passes only when T1–T9 pass on the pilot scope with the test event starting the clock on exactly the matching disposable asset and no effect outside the pilot. Expansion passes only when T3–T9 pass at production scope, the asset-ID / keyword matching data is present on production content, and the real raise process is tested against the system of record. Any fail on an **irreversible** action (regulatory record, or a real event that has started/elapsed a delete clock) halts the rollout pending records/legal review.

6. Troubleshooting

Common failure modes when implementing event-based retention in Microsoft Purview, and how to resolve them. Most retention behavior is **eventually consistent** — before treating something as broken, confirm it is not simply **propagation latency** (see the last item). Portal paths, the `Compliance Asset`

`Id` property, and cmdlet parameters are **plan- and cloud-dependent** — verify in your tenant/cloud.

Cannot connect / event or label cmdlets missing

- **Symptom:** `Connect-IPPSSession` fails, or `Get-ComplianceRetentionEventType` / `New-ComplianceTag` /

`New-ComplianceRetentionEvent` are "not recognized."

- **Likely cause:** Not connected to Security & Compliance PowerShell; the **ExchangeOnlineManagement**

module is missing/old; or the session lacks the licensing/scope for event-based retention.

- **Resolve:** Install/update `ExchangeOnlineManagement`; run `Connect-IPPSSession` (with `-ConnectionUri`

for GCC/GCC High/DoD). Run `Resolve-EventRetentionScope.ps1` to confirm connectivity, cloud, and reachable cmdlets. If the event cmdlets are absent, confirm the **E5 / E5 Compliance-class** licensing and the **Records Management / Compliance Administrator** role.

"Insufficient permissions" creating event types, labels, or raising events

- **Symptom:** Access-denied on `New-ComplianceRetentionEventType`, `New-ComplianceTag`, or

`New-ComplianceRetentionEvent`.

- **Likely cause:** The admin is not in the required Purview role group, or is not among the designated

event-raising operators.

- **Resolve:** Add the account to **Records Management** or **Compliance Administrator** /

Compliance Data Administrator (Purview portal → Roles & scopes → Permissions). Confirm the account is one of the authorized event-raisers (2.5). Re-check after assignment (role propagation can take time).

"Start retention based on an event" option / event-based label type is missing

- **Symptom:** When creating a label you cannot choose an event-based trigger, or `New-ComplianceTag` rejects

the event retention type.

- **Likely cause:** The tenant is not licensed for event-based retention, or the exact `-RetentionType`

value / parameter differs in your module version.

- **Resolve:** Confirm the **E5 / E5 Compliance-class** entitlement. Verify the correct event retention type

value for your tenant/module version (the design uses an `Event*` type) before scripting, and check the option in the portal first.

An event-based label doesn't appear for users (publish)

- **Symptom:** Pilot users don't see the label in SharePoint/OneDrive.

- **Likely cause:** Label-publish **propagation delay** (can take a day or more); the label policy's

locations don't include the user/site; or the client hasn't refreshed.

- **Resolve:** Confirm the label policy (`Label Policy – EventBased-Publish – Pilot`) targets the pilot

site/user; allow up to a few days; refresh the client. Verify with `Get-RetentionComplianceRule` that `PublishComplianceTag` names the correct label.

Auto-apply isn't labeling matching content

- **Symptom:** Content that should match an auto-apply condition stays unlabeled.
- **Likely cause:** Auto-apply runs on a service timer and can take **7+ days** for existing content; the

`ContentMatchQuery` / condition doesn't actually match; the location isn't in scope; or the plan doesn't include auto-apply.

- **Resolve:** Test the query against real content, confirm the location is in the policy, confirm licensing,

and **wait for the timer** before judging.

Labeled content shows "pending" and the clock never starts

- **Symptom:** Items with an event-based label stay in *"retention starts on event"* forever and never begin

counting.

- **Likely cause (most common): No retention event has been raised** for that event type — the label is

working exactly as designed, but the trigger is missing. Or an event was raised but its **asset ID / keyword didn't match**, or the label's `EventType` differs from the event's type.

- **Resolve:** Raise the event with `Set-EventTypesAndLabels.ps1 -RaiseEvent` (or `New-ComplianceRetentionEvent`)

using the **exact** asset ID / keyword that matches the content and the **same event type** the label uses. Confirm the label's `RetentionType` is an `Event*` type and its `EventType` name matches the event. Then allow for propagation. **If no one is raising events at all, fix the raise process (2.3) — that is the real defect.**

A raised event matched nothing (or the wrong content)

- **Symptom:** After raising an event, no clocks start, or clocks start on unexpected items.
- **Likely cause:** The `Compliance Asset Id` was never stamped on the target documents (asset-ID

matching had nothing to match); the asset ID supplied at raise time doesn't equal the one stamped; or a **keyword** query is too broad/narrow.

- **Resolve:** Confirm the `Compliance Asset Id` is present and correct on the in-scope content (T7), and

that the raise supplies the identical value. For keyword categories, validate the query matches in-scope and excludes out-of-scope content before re-raising. Remember matching data must exist **before** the event.

The clock started from the wrong date

- **Symptom:** Retention is counting from a date that isn't when the event actually occurred.

• **Likely cause:** The retention event was raised with the **raise date** rather than the true **event date** (`-EventDateTime`).

• **Resolve:** Set `-EventDateTime` to when the event actually happened (e.g. the real termination date), especially when raising after the fact. If already raised incorrectly, correct it per records/legal direction before the clock elapses.

An event-based label also declares a record and content is locked unexpectedly

• **Symptom:** After an event-based **record** label applies, owners lose edit/delete.

• **Likely cause:** The label was created with the record setting (by design) — a record locks content; a **regulatory** record is a stronger, one-way lock.

• **Resolve:** Confirm with `Get-ComplianceTag` whether `IsRecordLabel` / `Regulatory` is set; communicate the lock before applying record labels to live content. Records declaration/un-declaration mechanics are owned by the data lifecycle & records product — defer to it.

Retention differs from expectation — a retention conflict

• **Symptom:** The effective retention/deletion on an item doesn't match the event-based label you set.

• **Likely cause: Retention precedence** across overlapping policies/labels: retention wins over deletion; the **longest** retention wins; **explicit** (label) beats **implicit** (policy); the **shortest** deletion wins.

• **Resolve:** Enumerate every policy/label touching the content (`Get-EventRetentionPosture.ps1` plus the broader program's assessment), reason through precedence, and **redesign** the overlap to produce the intended effective outcome — do not force a result against the precedence rules.

Government-cloud portal path, property, or parameter differs / is missing

• **Symptom:** A portal path 404s, the `Compliance Asset Id` behaves differently, an endpoint fails, or a feature isn't present in GCC/GCC High/DoD.

• **Likely cause:** Sovereign-cloud differences — different URLs (`*.office365.us`), different `Connect-IPPSSession -ConnectionUri`, and some features/parameters lag or differ.

• **Resolve:** Use the correct sovereign endpoint and portal URL for your cloud, and **verify feature and property availability in your tenant** before relying on it. Treat any Commercial-vs-Government difference as a tenant-verify item, not an error.

It "isn't working" — but it's actually propagation latency

• **Symptom:** A just-made change (label publish, asset-ID stamp, a raised event) hasn't taken effect.

• **Likely cause:** Normal **eventual consistency** — label propagation, asset-ID indexing, and event processing run on service timers (hours to several days).

• **Resolve:** Wait the expected window before re-testing; don't stack "fixes" on top of a change that

simply hasn't propagated. Re-run `Get-EventRetentionReport.ps1` after the window to confirm state.

7. Best Practices

Durable practices for running event-based retention in Microsoft Purview. They encode the house model — pilot-first, evidence-based, records/legal-owned determinations — into day-to-day operation, and they concentrate on the part that actually fails in the field: getting an event to reliably start the clock on the right content.

Governance & ownership

- **Records/legal own the schedule; you own the implementation.** Never invent an event type, a post-event

retention period, a record status, or a matching rule. Treat the event-based schedule as the source of truth and keep a traceable link from each event type and label back to its schedule category and citation.

- **Keep a single, shared inventory.** Reuse existing event types and labels from the broader retention/

records program — one event type per real-world event, one label per category, named per the convention. Do not mint duplicate event types (`Employee-Termination` twice) — they fragment triggering and reporting.

- **Restrict who can raise events.** Raising a retention event starts real retention clocks. Limit

event-raising to a small, named, authorized set of operators and route every real event through change control; keep it separate from routine label administration.

The raise process is the deliverable

- **An event type without a raise process is dead weight.** The most common failure is content that sits

pending forever because no one raises the event. For every event type, name the **system of record** (HR/CLM/PLM), the **signal** that indicates the event, and the **owner** who raises it — and prefer **integrating** the raise with the system of record over relying on someone to remember.

- **Supply the right asset IDs / keywords at raise time.** The event only matches content you identify;

build the raise process to pull the exact asset IDs (or keyword) from the system of record so the clock starts on precisely the affected items.

- **Record the event date deliberately.** The retention clock counts from the **event date**, not the raise

date — set it to when the event actually occurred (e.g. the real termination date), especially if the event is raised after the fact.

Asset-ID / keyword matching

- **Stamp the asset ID before the event, ideally at ingestion.** Matching is only as good as the

`Compliance Asset Id` on the content. Bake asset-ID stamping into the content's creation/ingestion process so the data is already there when the event fires — retrofitting IDs after an event is error-prone.

- **Prefer asset IDs for precision; use keywords deliberately.** Asset-ID matching (SharePoint/OneDrive)

affects only tagged items; keyword matching (typically Exchange) is broader — validate a keyword query against real content to confirm it matches in-scope and **excludes** out-of-scope content before relying on it.

- **Keep asset-ID schemes stable and documented.** The identifier used to stamp content (employee ID /

contract number / SKU) must be the same value the raise process supplies — document the scheme and its source so the two never drift apart.

Change-safety

- **Publish to pilot, prove the trigger, then expand.** Labels are published to the pilot scope; the trigger

is proven by a **test event on a disposable asset**; production is a deliberate, change-controlled step after the gates pass.

- **Test events on disposable assets only.** Prove the clock start on a throwaway asset before wiring the

real process — never rehearse with content that a started (or elapsed) delete clock would harm.

- **Write the rollback — and flag the irreversible parts.** Reversible: unpublish/disable label policies,

remove non-record labels. Mitigable-before-elapse: extend/relabel while a delete clock still runs. **No-rollback:** an elapsed event-based deletion and any regulatory-record designation. Every change ticket states which is which.

Operations & evidence

- **Monitor pending vs triggered.** A growing population of **pending** event-based content with no

corresponding events being raised is an operational red flag — either events are not happening, or the raise process has broken. Report on it, don't just count labels.

- **Report on a cadence.** Run the compliance report regularly; watch event-type coverage, event-based label

counts, raised events, and gaps (labels with no event type, event types with no label, events that matched nothing).

- **Respect eventual consistency.** Label propagation, asset-ID indexing, and event processing run on timers

(hours to days). Build that latency into expectations and never judge a raised event in the first minutes.

- **Verify by cloud.** In GCC/GCC High/DoD, confirm portal paths, endpoints (`Connect-IPPSSession -ConnectionUri), the Compliance Asset Id` property, and cmdlet parameters — do not assume Commercial parity.

- **Keep automation credential-free.** All scripts use interactive modern auth; never embed secrets. Keep

assessment/validation/reporting read-only and configuration (including event-raising) `-WhatIf`-by-default with an explicit `-Commit`.

Alignment with the broader program

- **Design with retention precedence.** Where an event-based label overlaps other retention on the same

content, reason through Microsoft's precedence (retention > deletion; longest retention wins; explicit > implicit; shortest deletion wins) and **validate the effective outcome** — don't assume it.

- **Defer records, file plan, and disposition mechanics.** Records declaration in depth, the file plan, and

disposition-review operations belong to the org-wide data lifecycle & records product — align label names and defer those mechanics rather than re-documenting them.

- **Don't rebuild what the program owns.** Sensitivity labels, DLP, and tenant-wide data governance belong

to the broader Purview program — align to them, don't duplicate.

8. Deliverables Summary

Everything included in this solution, on one page.

Documentation

- **Solution Overview** — what event-based retention is and why the calendar cannot express it; the

event type → **event-based label** → **retention event** mechanic; **asset-ID vs keyword** triggering (the `Compliance Asset Id` property for SharePoint/OneDrive, keywords for Exchange); how content sits **pending** until an event is raised; the reference design; what's deferred to records/legal and to the broader data lifecycle & records product; limitations; and the deployment model.

- **Requirements Guide** — solution-specific prerequisites (compliance licensing for event-based retention/

records; the event-based schedule from records/legal; the **system of record and event-raise process**; the **asset-ID / keyword matching strategy and its data**; roles incl. event-raise authorization; in-scope workloads; alignment with the broader program; the S&C PowerShell / Graph toolchain; change control + rollback), each with how to verify and implement, building on the Purview category global requirements.

- **Implementation Guide** — Assess → Design → Configure → Pilot → Validate → Expand, with every artifact

named, event types / labels / label policies built in order, asset IDs stamped on pilot content, the trigger proven by a **test event on a disposable asset**, explicit pass/fail gates per stage, and a documented rollback that flags the **irreversible** parts (elapsed event-based deletions, regulatory records).

- **Testing & Validation** — 9 procedures, each with an explicit pass/fail, covering toolchain/scope,

baseline inventory, event types, label-to-event-type binding, publish/auto-apply scope, the pending state, asset-ID / keyword matching data, the test event starting the clock on exactly the matching content, and

reporting.

- **Troubleshooting** — common failure modes (connection/roles/plan, labels not appearing or applying, the

clock never starting, events matching nothing or too much, asset-ID not stamped, wrong event date, Government-cloud differences, propagation latency) and resolutions.

- **Best Practices** — governance/ownership, the raise process as the deliverable, asset-ID / keyword

matching, change-safety, operations & evidence, and alignment with the broader program.

- **Executive Summary** — the outcome, approach, and why it is low-risk, for decision-makers.

PowerShell scripts (5)

All use modern-auth `Connect-IPPSSession` (Security & Compliance PowerShell) and `Connect-MgGraph` where noted; **no hardcoded credentials**. Assessment, validation, and reporting are read-only; the configuration script (including event-raising) defaults to dry-run / `-WhatIf` and only writes when `-Commit` is passed; regulatory-record actions are gated behind an explicit `-AllowRegulatory -SignOffRef`, and raising a real event is a deliberate, gated action.

1. `Resolve-EventRetentionScope.ps1` — *Scope/health helper (read-only).* Checks S&C PowerShell (and

optional Graph) connectivity, reports the detected cloud/endpoint, confirms the event-based retention cmdlets (`Get-ComplianceRetentionEventType`, `Get-ComplianceTag`, `Get-ComplianceRetentionEvent`) are reachable, and resolves the pilot scope — the preflight before Stage 3.

1. `Get-EventRetentionPosture.ps1` — *Discovery (read-only).* Inventories existing **event types**,

existing **event-based labels** (those with an `Event*` retention type, with their `EventType` and record/regulatory flags), the **label policies** publishing/auto-applying them, and any **raised events**. Produces `Event Retention Posture – Tenant – Discovery` (CSVs), flagging duplicates and orphans.

1. `Set-EventTypesAndLabels.ps1` — *Configuration/remediation helper (`-WhatIf` default).* Creates/verifies

event types (`New-ComplianceRetentionEventType`) and **event-based labels** (`New-ComplianceTag -RetentionType EventAgeInDays -EventType …`) from a JSON design, and — with `-RaiseEvent` — raises a retention event (`New-ComplianceRetentionEvent`) against asset IDs / keywords. Dry-run by default; writes only with `-Commit`; refuses regulatory labels without `-AllowRegulatory -SignOffRef`; test events on disposable assets.

1. `Test-EventRetentionConfig.ps1` — *Validation (read-only).* Checks each designed event type exists,

each label uses an **event** trigger bound to the **correct** event type, label policies target the pilot scope, and reports asset-ID / keyword readiness and linkage gaps — the Stage 5 gate check. Makes no changes.

1. `Get-EventRetentionReport.ps1` — *Compliance report (read-only).* Compiles event-type coverage,

event-based label coverage (event type / record / regulatory / action / duration), and raised events into `Event Retention Compliance – Tenant – Report` (CSV + HTML), flagging gaps (labels with no event type, event types with no label, events that matched nothing) and pointing to the portal for disposition.

Appendix A — PowerShell Scripts

Included in the accompanying ZIP. Each script's header documents full usage.

Get-EventRetentionPosture.ps1

Get-EventRetentionReport.ps1

Resolve-EventRetentionScope.ps1

Set-EventTypesAndLabels.ps1

Test-EventRetentionConfig.ps1